

osu! Song Exporter + Spotify Playlist Sync

Project overview, implementation brief, and AI-agent prompt

Prepared for a Python-based automation project

Core idea: Collect songs from osu! activity and/or local beatmap files, export them into structured data, then optionally match them to Spotify tracks and add confident matches to a playlist.

Goal	Feasible?	Best Source
Songs recently played	Yes	osu! API: GET /users/{user}/scores/recent
Best plays	Yes	osu! API: GET /users/{user}/scores/best
Most played beatmaps	Yes	osu! API: GET /users/{user}/beatmapsets/most_played
Favourite beatmaps	Yes	osu! API: GET /users/{user}/beatmapsets/favourite
Every downloaded beatmap/song	Not cleanly via API	Parse local osu! Songs folder and .osu files
Export list with osu! links	Yes	API data and local parser
Add matches to Spotify playlist	Yes, with caveats	Spotify Web API via Spotipy

1. Project Goal

Build a Python project that collects songs from a user's osu! activity and/or local osu! installation, exports them into a structured list, and optionally matches them to Spotify tracks and adds them to a Spotify playlist.

- Collect recently played, best, most-played, and favourite beatmaps via osu! API v2.
- Collect locally downloaded beatmaps by scanning the osu! Songs folder and parsing .osu files.
- Deduplicate multiple difficulties from the same beatmapset.
- Export CSV, JSON, or Markdown files with osu! links.
- Search Spotify for matching tracks and add high-confidence matches to a playlist.
- Save uncertain or unmatched tracks for manual review.

Important reality check: The osu! API is useful for public/user activity, but it does not expose a reliable endpoint for all beatmaps downloaded on a specific PC. For downloaded beatmaps, parse the local Songs folder.

2. Recommended Tech Stack

Component	Recommendation
Language	Python 3.11+
osu! API client	requests, httpx, or a wrapper such as ossapi / aiosu
Spotify API client	spotipy
Configuration	python-dotenv
Export formats	csv, json, pathlib
Fuzzy matching	rapidfuzz
CLI	argparse, typer, or click

```
pip install requests spotipy python-dotenv rapidfuzz
# Optional CLI helpers:
pip install typer rich
```

3. External APIs

3.1 osu! API v2

Item	Details
Base URL	https://osu.ppy.sh/api/v2
OAuth token URL	POST https://osu.ppy.sh/oauth/token
Recommended auth for public data	OAuth client credentials with scope=public
Rate limit note	The docs currently state 1200 requests per minute; still cache and avoid wasteful requests.

3.2 osu! Auth - Client Credentials

Use this for public data such as user profile, public scores, favourites, and most-played beatmaps.

```
POST https://osu.ppy.sh/oauth/token
Content-Type: application/x-www-form-urlencoded
Accept: application/json
```

```
client_id=<OSU_CLIENT_ID>
client_secret=<OSU_CLIENT_SECRET>
grant_type=client_credentials
scope=public
```

```
{
  "access_token": "...",
  "expires_in": 86400,
  "token_type": "Bearer"
}
```

3.3 osu! Auth - Authorization Code

Use this only if the app needs authenticated user data through endpoints such as /me/{mode?}.

```
POST https://osu.ppy.sh/oauth/token
```

```
client_id=<OSU_CLIENT_ID>
client_secret=<OSU_CLIENT_SECRET>
code=<AUTHORIZATION_CODE>
grant_type=authorization_code
redirect_uri=<REDIRECT_URI>
```

4. osu! API Endpoints to Implement

Use Case	Endpoint	Notes
Resolve user	GET /users/{user}/{mode?}	user can be ID or @username; mode can be osu, taiko, fruits, mania
Get own user	GET /me/{mode?}	Requires user OAuth with identify scope
Recent scores	GET /users/{user}/scores/recent	Use mode=osu, include_fails=1, limit=50
Best scores	GET /users/{user}/scores/best	Use mode=osu, limit=100
First-place scores	GET /users/{user}/scores/firsts	Use mode=osu, limit=100
Favourite beatmaps	GET /users/{user}/beatmapsets/favourite	Returns beatmapsets
Most played beatmaps	GET /users/{user}/beatmapsets/most_played	Returns BeatmapPlaycount objects
Recent activity	GET /users/{user}/recent_activity	Less useful for clean song extraction

4.1 Get User

```
GET https://osu.ppy.sh/api/v2/users/@username/osu
```

Use this to resolve a username to user metadata. Prefix usernames with @ when using a username instead of an ID.

4.2 Get User Scores

```
GET /users/{user}/scores/{type}
```

Valid type values:

- best
- firsts
- recent

Examples:

```
GET /users/{user}/scores/recent?mode=osu&include_fails=1&limit=50
GET /users/{user}/scores/best?mode=osu&limit=100
GET /users/{user}/scores/firsts?mode=osu&limit=100
```

Parameter	Purpose
legacy_only	Optional integer; defaults to 0
include_fails	Only for recent scores; set to 1 to include failed plays
mode	Ruleset: osu, taiko, fruits, mania
limit	Maximum number of results
offset	Pagination offset

4.3 Extracted Score Fields

```
score["beatmapset"]["id"]
score["beatmapset"]["artist"]
score["beatmapset"]["artist_unicode"]
score["beatmapset"]["title"]
score["beatmapset"]["title_unicode"]
score["beatmap"]["id"]
```

```
osu beatmapset link: https://osu.ppy.sh/beatmapsets/{beatmapset_id}
osu beatmap link:   https://osu.ppy.sh/beatmaps/{beatmap_id}
```

4.4 Get User Beatmaps

```
GET /users/{user}/beatmapsets/{type}
```

Useful type values:

- favourite
- most_played
- ranked
- loved
- pending
- graveyard
- guest
- nominated

Examples:

```
GET /users/{user}/beatmapsets/favourite?limit=100
GET /users/{user}/beatmapsets/most_played?limit=100
```

Important: type=most_played returns BeatmapPlaycount objects, while most other types return BeatmapsetExtended objects. Implement parsing defensively.

5. Local osu! Song Folder Parsing

Use this path for classic osu! on Windows:

```
C:\Users\<username>\AppData\Local\osu!\Songs
```

Each beatmapset folder usually contains one or more .osu files, one per difficulty. Parse the [General] and [Metadata] sections.

Section	Fields to Extract
[General]	AudioFilename, PreviewTime, Mode

[Metadata]	Title, TitleUnicode, Artist, ArtistUnicode, Creator, Version, Source, Tags, BeatmapID, BeatmapSetID
------------	---

```
title = TitleUnicode or Title
artist = ArtistUnicode or Artist
```

```
deduplication key:
- BeatmapSetID if valid and not -1
- otherwise normalized artist + title
```

Deduplication: Do not treat every .osu file as a separate Spotify song. Multiple .osu files often represent different difficulties of the same beatmapset.

6. Spotify Web API

Item	Details
Base URL	https://api.spotify.com/v1
Recommended wrapper	spotipy
Auth flow	Authorization Code via SpotifyOAuth
Required scopes	playlist-modify-private playlist-modify-public

```
SPOTIPY_CLIENT_ID=...
SPOTIPY_CLIENT_SECRET=...
SPOTIPY_REDIRECT_URI=http://localhost:8888/callback
```

6.1 Create Playlist

```
POST /me/playlists
```

```
{
  "name": "osu! imports",
  "public": false,
  "description": "Generated from osu! beatmaps."
}
```

6.2 Search Tracks

```
GET /search
```

```
Recommended query patterns:
q=track:{title} artist:{artist}&type=track&limit=5
q={artist} {title}&type=track&limit=5
```

6.3 Add Items to Playlist

```
POST /playlists/{playlist_id}/items
```

```
{
  "uris": [
    "spotify:track:..."
  ]
}
```

Batch additions in chunks of at most 100 URIs.

7. Data Models

```
from dataclasses import dataclass

@dataclass
class OsuSong:
    source: str # osu_api_recent, osu_api_best, osu_api_favourite, local
    artist: str
    title: str
    artist_romanized: str | None
    title_romanized: str | None
    beatmapset_id: int | None
    beatmap_id: int | None
    audio_filename: str | None
    creator: str | None
    difficulty: str | None
    osu_beatmapset_url: str | None
    osu_beatmap_url: str | None

@dataclass
class SpotifyMatch:
    osu_song_key: str
    spotify_track_id: str | None
    spotify_uri: str | None
    spotify_url: str | None
    matched_artist: str | None
    matched_title: str | None
    confidence: float
    status: str # matched, review, unmatched
```

8. Suggested Project Structure

```
osu_spotify_sync/
├── README.md
├── pyproject.toml
├── .env.example
├── src/
│   └── osu_spotify_sync/
│       ├── __init__.py
│       ├── config.py
│       ├── models.py
│       ├── osu_api.py
│       ├── local_osu.py
│       ├── normalize.py
│       ├── spotify_api.py
│       ├── matching.py
│       ├── export.py
│       └── cli.py
├── exports/
│   ├── osu_songs.csv
│   ├── osu_songs.json
│   ├── spotify_matches.csv
│   └── spotify_unmatched.csv
└── tests/
    ├── test_local_osu.py
    ├── test_normalize.py
    └── test_matching.py
```

9. Environment Variables

OSU_CLIENT_ID=

osu! Song Exporter + Spotify Playlist Sync

```
OSU_CLIENT_SECRET=

SPOTIFY_CLIENT_ID=
SPOTIFY_CLIENT_SECRET=
SPOTIFY_REDIRECT_URI=http://localhost:8888/callback

OSU_USERNAME=
OSU_USER_ID=
OSU_RULESET=osu
OSU_SONGS_PATH=C:\Users\<username>\AppData\Local\osu!\Songs
```

10. Core Features and CLI Commands

10.1 MVP 1 - Local Export Only

```
python -m osu_spotify_sync scan-local --songs-path "C:\...\osu!\Songs" --out exports/osu_songs.csv
```

```
Output columns:
source
artist
title
artist_romanized
title_romanized
beatmapset_id
beatmap_id
creator
difficulty
audio_filename
osu_beatmapset_url
osu_beatmap_url
```

10.2 MVP 2 - osu! API Import

```
python -m osu_spotify_sync fetch-osu --user "@username" --type recent --out exports/recent.csv
python -m osu_spotify_sync fetch-osu --user "@username" --type best --out exports/best.csv
python -m osu_spotify_sync fetch-osu --user "@username" --type favourite --out exports/favourites.csv
python -m osu_spotify_sync fetch-osu --user "@username" --type most_played --out exports/most_played.csv
```

```
Command mapping:
recent      -> GET /users/{user}/scores/recent
best        -> GET /users/{user}/scores/best
firsts      -> GET /users/{user}/scores/firsts
favourite   -> GET /users/{user}/beatmapsets/favourite
most_played -> GET /users/{user}/beatmapsets/most_played
```

10.3 MVP 3 - Spotify Matching

```
python -m osu_spotify_sync match-spotify --input exports/osu_songs.csv --out exports/spotify_matches.csv
```

1. Normalize title and artist.
2. Remove common osu!/anime metadata noise such as TV Size, Short Ver., Game Ver., Cut Ver., feat., and bracketed annotations.
3. Search Spotify using strict query first, then fallback query.
4. Score candidates using fuzzy title similarity, artist similarity, and optionally duration similarity.
5. Save matched, review, and unmatched results separately.

10.4 MVP 4 - Playlist Creation

```
python -m osu_spotify_sync create-playlist --matches exports/spotify_matches.csv --playlist-name "osu! imports" --private
```

- Only add rows with status=matched.
- Deduplicate Spotify URIs before adding.
- Batch additions in chunks of 100.
- Print final playlist URL.
- Keep review and unmatched tracks as CSV files.

11. Matching Strategy Details

Main weak point: osu! metadata often does not match Spotify metadata exactly. The implementation should never blindly add low-confidence matches.

Problem	Example
Anime OP/ED metadata	The beatmap title may be the song title while Spotify release title differs.
TV size versions	Spotify may only have the full version, or not the TV size.
Romanization differences	osu! may use Japanese Unicode while Spotify uses romanized text, or vice versa.
Remixes and covers	Nightcore, bootlegs, covers, or mapper edits may not exist on Spotify.
Extra annotations	Titles may include difficulty names, feat. text, brackets, or mapper annotations.

Suggested confidence rule:
 $\text{score} = 0.6 * \text{title_similarity} + 0.3 * \text{artist_similarity} + 0.1 * \text{duration_similarity}$

```
if score >= 85:
    status = "matched"
elif score >= 70:
    status = "review"
else:
    status = "unmatched"
```

```
def clean_title(title: str) -> str:
    remove_patterns = [
        r"\(TV Size\)",
        r"\[TV Size\]",
        r"\(Short Ver\.\?\)",
        r"\(Game Ver\.\?\)",
        r"\(Cut Ver\.\?\)",
        r"\(feat\.\.*\?)",
        r"\[.*?\]",
    ]
    # Apply carefully and keep original metadata as well.
```

12. Export Formats

File	Purpose
exports/osu_songs.csv	Main list of extracted osu! songs and metadata
exports/osu_songs.json	Optional structured JSON export

exports/spotify_matches.csv	Spotify search results and confidence scores
exports/spotify_unmatched.csv	Tracks that could not be matched confidently

```
osu_songs.csv columns:
source, artist, title, artist_romanized, title_romanized,
beatmapset_id, beatmap_id, creator, difficulty,
audio_filename, osu_beatmapset_url, osu_beatmap_url

spotify_matches.csv columns:
artist, title, spotify_artist, spotify_title, spotify_uri,
spotify_url, confidence, status, osu_beatmapset_url

spotify_unmatched.csv columns:
artist, title, reason, osu_beatmapset_url
```

13. Error Handling Checklist

- Missing osu! credentials
- Expired osu! access token
- Invalid osu! username
- Private, deleted, or unavailable beatmapsets
- Missing or invalid BeatmapSetID
- Invalid local osu! path
- Malformed .osu files
- Encoding issues in .osu files
- Spotify authentication failure
- Spotify rate limits
- Spotify search returns no results
- Spotify playlist add fails
- Duplicate Spotify URIs

```
path.read_text(encoding="utf-8-sig", errors="ignore")
```

14. Implementation Plan for Another AI Agent

Stage	Files	Work
1	models.py, local_osu.py, export.py, cli.py	Implement local scanning, deduplication, and CSV export.
2	osu_api.py	Implement token retrieval, user lookup, scores, favourites, and most-played import.
3	spotify_api.py, matching.py	Implement Spotify search, fuzzy scoring, review/unmatched output.
4	cli.py, tests/	Add CLI commands and unit tests.

15. Direct Prompt for Another AI Agent

You are building a Python 3.11+ project called `osu_spotify_sync`.

Goal:
Create a CLI tool that collects song metadata from osu! and optionally syncs matched songs into a Spotify playlist.

osu! Song Exporter + Spotify Playlist Sync

Data sources:

1. Local osu! Songs folder:

- Recursively scan for .osu files.
- Parse [General] and [Metadata] sections.
- Extract AudioFilename, Title, TitleUnicode, Artist, ArtistUnicode, Creator, Version, Source, Tags, BeatmapID, BeatmapSetID.
- Prefer Unicode fields when available.
- Deduplicate by valid BeatmapSetID; fallback to normalized artist-title key.
- Generate osu! links:
https://osu.ppy.sh/beatmapsets/{BeatmapSetID}
https://osu.ppy.sh/beatmaps/{BeatmapID}

2. osu! API v2:

Base URL: https://osu.ppy.sh/api/v2

Token URL: POST https://osu.ppy.sh/oauth/token

Use client credentials for public data:

client_id=<OSU_CLIENT_ID>

client_secret=<OSU_CLIENT_SECRET>

grant_type=client_credentials

scope=public

Implement endpoints:

- GET /users/{user}/{mode?}
- GET /me/{mode?}
- GET /users/{user}/scores/{type}
where type is best, firsts, or recent.
Useful params: legacy_only, include_fails, mode, limit, offset.
- GET /users/{user}/beatmapsets/{type}
useful types: favourite, most_played, ranked, loved, pending, graveyard, guest, nominated.
Useful params: limit, offset.

Spotify:

Use Spotipy.

Required scopes:

playlist-modify-private playlist-modify-public

Implement:

- Search tracks with Spotify /search using type=track and limit=5.
- Try strict query first: track:"{title}" artist:"{artist}"
- Then fallback: {artist} {title}
- Score candidates using fuzzy title similarity, artist similarity, and optionally duration similarity.
- Auto-accept confidence >= 85.
- Mark 70-84 as review.
- Mark below 70 or no result as unmatched.
- Create playlist with POST /me/playlists.
- Add items using POST /playlists/{playlist_id}/items.
- Batch additions in chunks of max 100 items.

Project structure:

```
osu_spotify_sync/  
├── src/osu_spotify_sync/  
│   ├── config.py  
│   ├── models.py  
│   ├── local_osu.py  
│   ├── osu_api.py  
│   ├── spotify_api.py  
│   ├── normalize.py  
│   ├── matching.py  
│   ├── export.py  
│   └── cli.py  
├── exports/  
├── tests/  
├── .env.example  
└── README.md
```

CLI commands:

- scan-local --songs-path PATH --out exports/osu_songs.csv

```
- fetch-osu --user USER --type recent|best|firsts|favourite|most_played --out FILE
- match-spotify --input FILE --out exports/spotify_matches.csv
- create-playlist --matches exports/spotify_matches.csv --playlist-name NAME --private
```

Implementation priorities:

1. Local scanner + CSV export.
2. osu! API importer.
3. Spotify matching with review/unmatched CSV.
4. Playlist creation and batched item adding.

Be careful:

- osu! API does not provide a clean all-downloaded-beatmaps-on-this-machine endpoint. Use local file parsing for downloaded beatmaps.
- Multiple .osu files can represent different difficulties of the same song. Deduplicate by BeatmapSetID.
- osu! metadata often differs from Spotify metadata. Never blindly add low-confidence matches.
- Keep original title/artist and cleaned title/artist.
- Use .env for credentials.
- Do not hardcode secrets.

16. References

- osu! API documentation: <https://osu.ppy.sh/docs/>
- osu! .osu file format documentation:
https://osu.ppy.sh/wiki/en/Client/File_formats/osu_%28file_format%29
- Spotify Web API scopes: <https://developer.spotify.com/documentation/web-api/concepts/scopes>
- Spotify Create Playlist endpoint:
<https://developer.spotify.com/documentation/web-api/reference/create-playlist>
- Spotify Search endpoint: <https://developer.spotify.com/documentation/web-api/reference/search>
- Spotify Add Items to Playlist endpoint:
<https://developer.spotify.com/documentation/web-api/reference/add-items-to-playlist>
- Spotify documentation: <https://spotify.readthedocs.io/>